SENG 201
Software Engineering Foundations



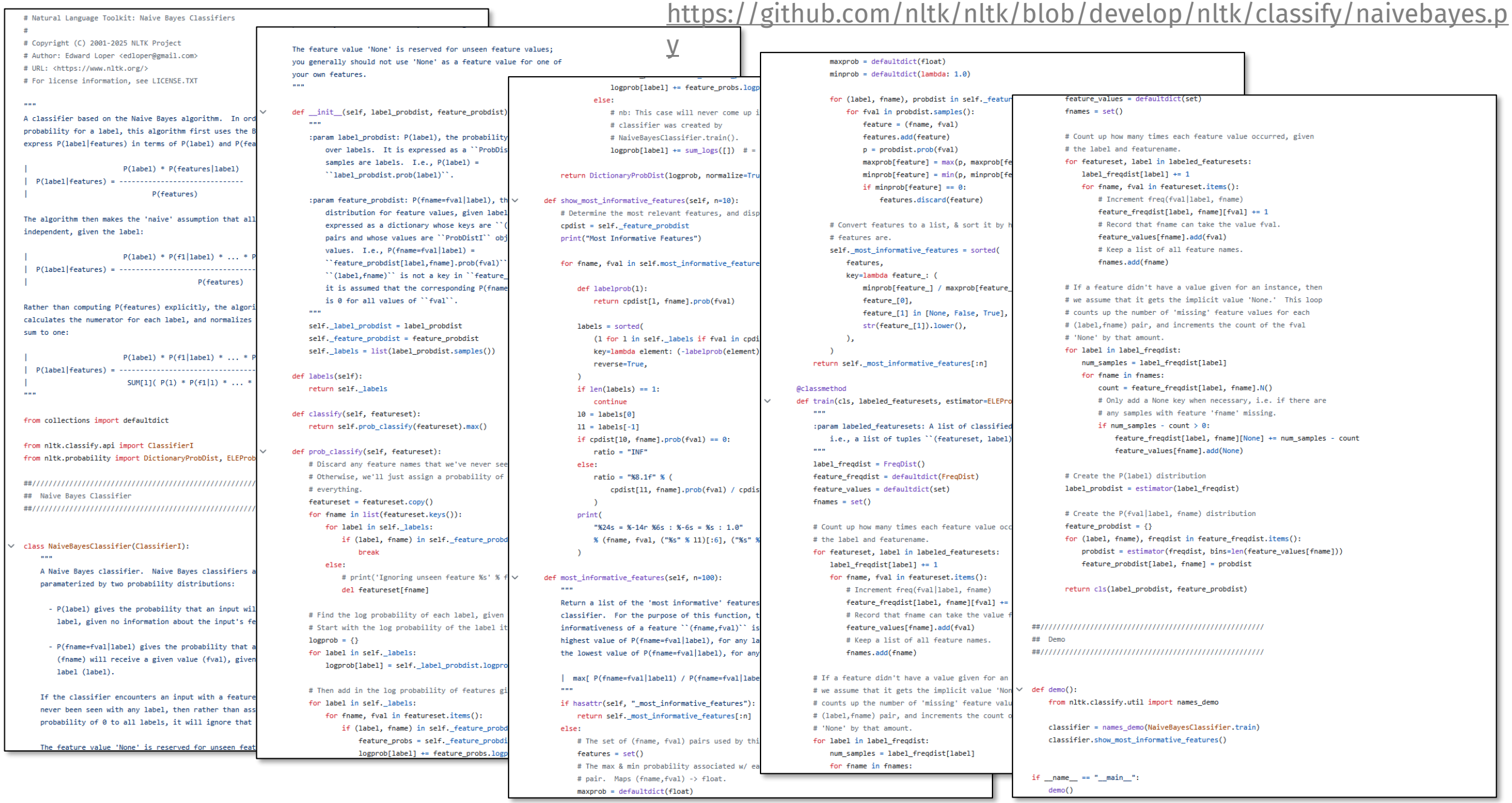
Developing software is just writing code… right?

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

```python
def total_cost(unit_price, nb_units):
    total = nb_units * unit_price
    if nb_units > 100:
        total = total * 0.9
    return total
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

https://github.com/nltk/nltk/blob/develop/nltk/classify/naivebayes.py



```
# Natural Language Toolkit: Naive Bayes Classifiers
#
# Copyright (C) 2001-2025 NLTK Project
# Author: Edward Loper <edloper@gmail.com>
# URL: <https://www.nltk.org/>
# For license information, see LICENSE.TXT

"""
A classifier based on the Naive Bayes algorithm.  In ord
probability for a label, this algorithm first uses the B
express P(label|features) in terms of P(label) and P(fea

|                        P(label) * P(features|label)
|  P(label|features) = ------------------------------
|                             P(features)

The algorithm then makes the 'naive' assumption that all
independent, given the label:

|                        P(label) * P(f1|label) * ... * P
|  P(label|features) = ----------------------------------
|                                        P(features)

Rather than computing P(features) explicitly, the algori
calculates the numerator for each label, and normalizes
sum to one:

|                        P(label) * P(f1|label) * ... * P
|  P(label|features) = ----------------------------------
|                         SUM[l]( P(l) * P(f1|l) * ... *
"""

from collections import defaultdict

from nltk.classify.api import ClassifierI
from nltk.probability import DictionaryProbDist, ELEProb

##//////////////////////////////////////////////////////
##  Naive Bayes Classifier
##//////////////////////////////////////////////////////


class NaiveBayesClassifier(ClassifierI):
    """
    A Naive Bayes classifier.  Naive Bayes classifiers a
    paramaterized by two probability distributions:

      - P(label) gives the probability that an input wil
        label, given no information about the input's fe

      - P(fname=fval|label) gives the probability that a
        (fname) will receive a given value (fval), given
        label (label).

    If the classifier encounters an input with a feature
    never been seen with any label, then rather than ass
    probability of 0 to all labels, it will ignore that

    The feature value 'None' is reserved for unseen feat
    you generally should not use 'None' as a feature value for one of
    your own features.
    """

    def __init__(self, label_probdist, feature_probdist)
        """
        :param label_probdist: P(label), the probability
            over labels.  It is expressed as a ``ProbDis
            samples are labels.  I.e., P(label) =
            ``label_probdist.prob(label)``.

        :param feature_probdist: P(fname=fval|label), th
            distribution for feature values, given label
            expressed as a dictionary whose keys are ``(
            pairs and whose values are ``ProbDistI`` obj
            values.  I.e., P(fname=fval|label) =
            ``feature_probdist[label,fname].prob(fval)``
            ``(label,fname)`` is not a key in ``feature_
            it is assumed that the corresponding P(fname
            is 0 for all values of ``fval``.
        """
        self._label_probdist = label_probdist
        self._feature_probdist = feature_probdist
        self._labels = list(label_probdist.samples())

    def labels(self):
        return self._labels

    def classify(self, featureset):
        return self.prob_classify(featureset).max()

    def prob_classify(self, featureset):
        # Discard any feature names that we've never see
        # Otherwise, we'll just assign a probability of
        # everything.
        featureset = featureset.copy()
        for fname in list(featureset.keys()):
            for label in self._labels:
                if (label, fname) in self._feature_probd
                    break
            else:
                # print('Ignoring unseen feature %s' % f
                del featureset[fname]

        # Find the log probability of each label, given
        # Start with the log probability of the label it
        logprob = {}
        for label in self._labels:
            logprob[label] = self._label_probdist.logpro

        # Then add in the log probability of features gi
        for label in self._labels:
            for fname, fval in featureset.items():
                if (label, fname) in self._feature_probd
                    feature_probs = self._feature_probdi
                    logprob[label] += feature_probs.logp
                else:
                    # nb: This case will never come up i
                    # classifier was created by
                    # NaiveBayesClassifier.train().
                    logprob[label] += sum_logs([])  # =

        return DictionaryProbDist(logprob, normalize=Tru

    def show_most_informative_features(self, n=10):
        # Determine the most relevant features, and disp
        cpdist = self._feature_probdist
        print("Most Informative Features")

        for fname, fval in self.most_informative_feature

            def labelprob(l):
                return cpdist[l, fname].prob(fval)

            labels = sorted(
                (l for l in self._labels if fval in cpdi
                key=lambda element: (-labelprob(element)
                reverse=True,
            )
            if len(labels) == 1:
                continue
            l0 = labels[0]
            l1 = labels[-1]
            if cpdist[l0, fname].prob(fval) == 0:
                ratio = "INF"
            else:
                ratio = "%8.1f" % (
                    cpdist[l1, fname].prob(fval) / cpdis
                )
            print(
                "%24s = %-14r %6s : %-6s = %s : 1.0"
                % (fname, fval, ("%s" % l1)[:6], ("%s" %
            )

    def most_informative_features(self, n=100):
        """
        Return a list of the 'most informative' features
        classifier.  For the purpose of this function, t
        informativeness of a feature ``(fname,fval)`` is
        highest value of P(fname=fval|label), for any la
        the lowest value of P(fname=fval|label), for any

        |  max[ P(fname=fval|label1) / P(fname=fval|labe
        """
        if hasattr(self, "_most_informative_features"):
            return self._most_informative_features[:n]
        else:
            # The set of (fname, fval) pairs used by thi
            features = set()
            # The max & min probability associated w/ ea
            # pair.  Maps (fname,fval) -> float.
            maxprob = defaultdict(float)
            minprob = defaultdict(lambda: 1.0)

            for (label, fname), probdist in self._featur
                for fval in probdist.samples():
                    feature = (fname, fval)
                    features.add(feature)
                    p = probdist.prob(fval)
                    maxprob[feature] = max(p, maxprob[fe
                    minprob[feature] = min(p, minprob[fe
                    if minprob[feature] == 0:
                        features.discard(feature)

            # Convert features to a list, & sort it by h
            # features are.
            self._most_informative_features = sorted(
                features,
                key=lambda feature_: (
                    minprob[feature_] / maxprob[feature_
                    feature_[0],
                    feature_[1] in [None, False, True],
                    str(feature_[1]).lower(),
                ),
            )
        return self._most_informative_features[:n]

    @classmethod
    def train(cls, labeled_featuresets, estimator=ELEPro
        """
        :param labeled_featuresets: A list of classified
            i.e., a list of tuples ``(featureset, label)
        """
        label_freqdist = FreqDist()
        feature_freqdist = defaultdict(FreqDist)
        feature_values = defaultdict(set)
        fnames = set()

        # Count up how many times each feature value occ
        # the label and featurename.
        for featureset, label in labeled_featuresets:
            label_freqdist[label] += 1
            for fname, fval in featureset.items():
                # Increment freq(fval|label, fname)
                feature_freqdist[label, fname][fval] +=
                # Record that fname can take the value f
                feature_values[fname].add(fval)
                # Keep a list of all feature names.
                fnames.add(fname)

        # If a feature didn't have a value given for an
        # we assume that it gets the implicit value 'Non
        # counts up the number of 'missing' feature valu
        # (label,fname) pair, and increments the count o
        # 'None' by that amount.
        for label in label_freqdist:
            num_samples = label_freqdist[label]
            for fname in fnames:
                count = feature_freqdist[label, fname].N()
                # Only add a None key when necessary, i.e. if there are
                # any samples with feature 'fname' missing.
                if num_samples - count > 0:
                    feature_freqdist[label, fname][None] += num_samples - count
                    feature_values[fname].add(None)

        # Create the P(label) distribution
        label_probdist = estimator(label_freqdist)

        # Create the P(fval|label, fname) distribution
        feature_probdist = {}
        for (label, fname), freqdist in feature_freqdist.items():
            probdist = estimator(freqdist, bins=len(feature_values[fname]))
            feature_probdist[label, fname] = probdist

        return cls(label_probdist, feature_probdist)


##//////////////////////////////////////////////////////
##  Demo
##//////////////////////////////////////////////////////

def demo():
    from nltk.classify.util import names_demo

    classifier = names_demo(NaiveBayesClassifier.train)
    classifier.show_most_informative_features()


if __name__ == "__main__":
    demo()
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

From writing code...





... to building software

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Developing software requires to...

Write code

```
float Q_rsqrt( float number )
{
    long i;
    float x2, y;
    const float threehalfs = 1.5F;

    x2 = number * 0.5F;
    y  = number;
    i  = * ( long * ) &y;                       // evil floating point bit level hacking
    i  = 0x5f3759df - ( i >> 1 );               // what the fuck?
    y  = * ( float * ) &i;
    y  = y * ( threehalfs - ( x2 * y * y ) );   // 1st iteration
//  y  = y * ( threehalfs - ( x2 * y * y ) );   // 2nd iteration, this can be removed

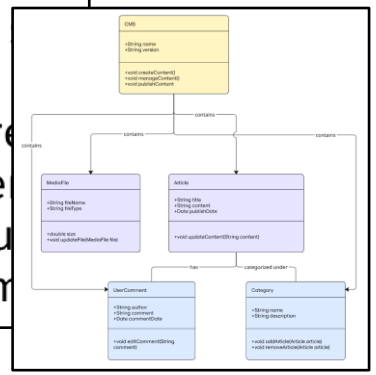
    return y;
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Developing software requires to...

Plan and Design

Requirements
- Be good
- Be fast
- Be secure
- Help use
- Starts qu
- Low mem



Write code

```
float Q_rsqrt( float number )
{
    long i;
    float x2, y;
    const float threehalfs = 1.5F;

    x2 = number * 0.5F;
    y  = number;
    i  = * ( long * ) &y;                       // evil floating point bit level hacking
    i  = 0x5f3759df - ( i >> 1 );               // what the fuck?
    y  = * ( float * ) &i;
    y  = y * ( threehalfs - ( x2 * y * y ) );   // 1st iteration
//  y  = y * ( threehalfs - ( x2 * y * y ) );   // 2nd iteration, this can be removed

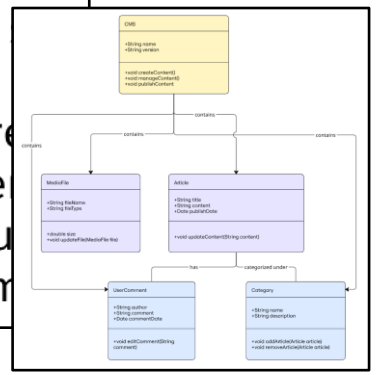
    return y;
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Developing software requires to...

Plan and Design

Requirements
- Be good
- Be fast
- Be secure
- Help use
- Starts qu
- Low mem



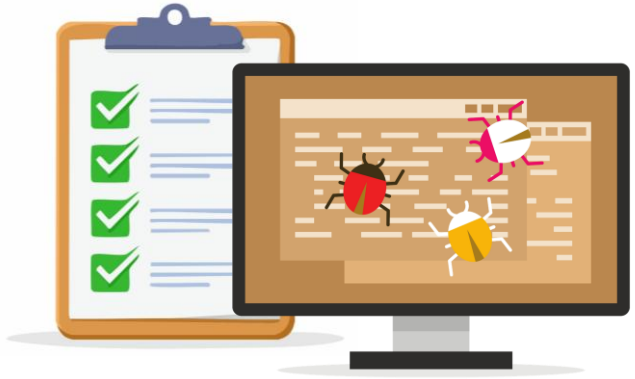
Write code

```
float Q_rsqrt( float number )
{
    long i;
    float x2, y;
    const float threehalfs = 1.5F;

    x2 = number * 0.5F;
    y  = number;
    i  = * ( long * ) &y;                       // evil floating point bit level hacking
    i  = 0x5f3759df - ( i >> 1 );               // what the fuck?
    y  = * ( float * ) &i;
    y  = y * ( threehalfs - ( x2 * y * y ) );   // 1st iteration
//  y  = y * ( threehalfs - ( x2 * y * y ) );   // 2nd iteration, this can be removed

    return y;
}
```

Test and Maintain

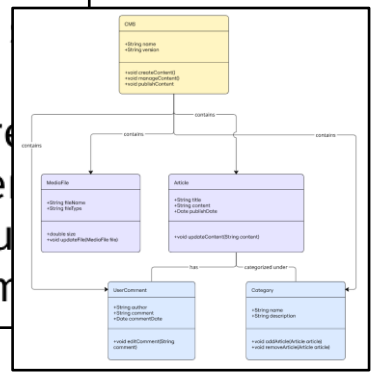


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Developing software requires to…

Plan and Design

Requirements
- Be good
- Be fast
- Be secure
- Help use
- Starts qu
- Low mem



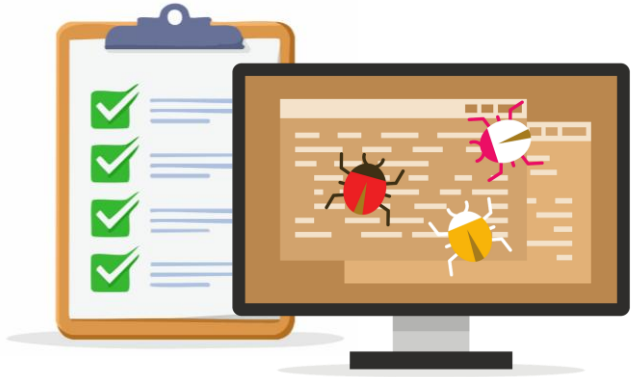
Write code

```
float Q_rsqrt( float number )
{
    long i;
    float x2, y;
    const float threehalfs = 1.5F;

    x2 = number * 0.5F;
    y  = number;
    i  = * ( long * ) &y;                       // evil floating point bit level hacking
    i  = 0x5f3759df - ( i >> 1 );               // what the fuck?
    y  = * ( float * ) &i;
    y  = y * ( threehalfs - ( x2 * y * y ) );   // 1st iteration
//  y  = y * ( threehalfs - ( x2 * y * y ) );   // 2nd iteration, this can be removed

    return y;
}
```

Test and Maintain



Package and Deploy

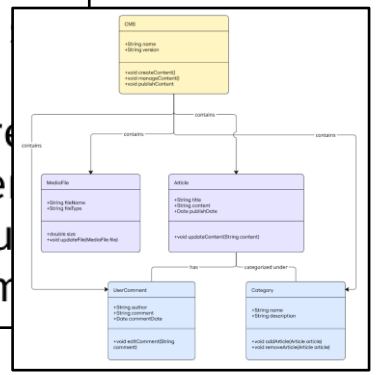


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

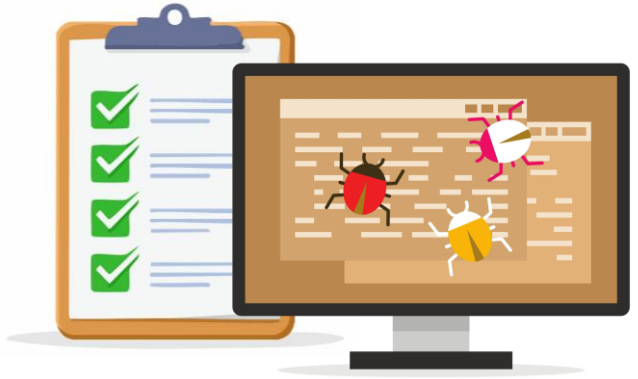
Developing software requires to...

Plan and Design

Requirements
- Be good
- Be fast
- Be secure
- Help use
- Starts qu
- Low mem



Test and Maintain



Write code

```
float Q_rsqrt( float number )
{
    long i;
    float x2, y;
    const float threehalfs = 1.5F;

    x2 = number * 0.5F;
    y  = number;
    i  = * ( long * ) &y;                       // evil floating point bit level hacking
    i  = 0x5f3759df - ( i >> 1 );               // what the fuck?
    y  = * ( float * ) &i;
    y  = y * ( threehalfs - ( x2 * y * y ) );   // 1st iteration
//  y  = y * ( threehalfs - ( x2 * y * y ) );   // 2nd iteration, this can be removed

    return y;
}
```

Collaborate



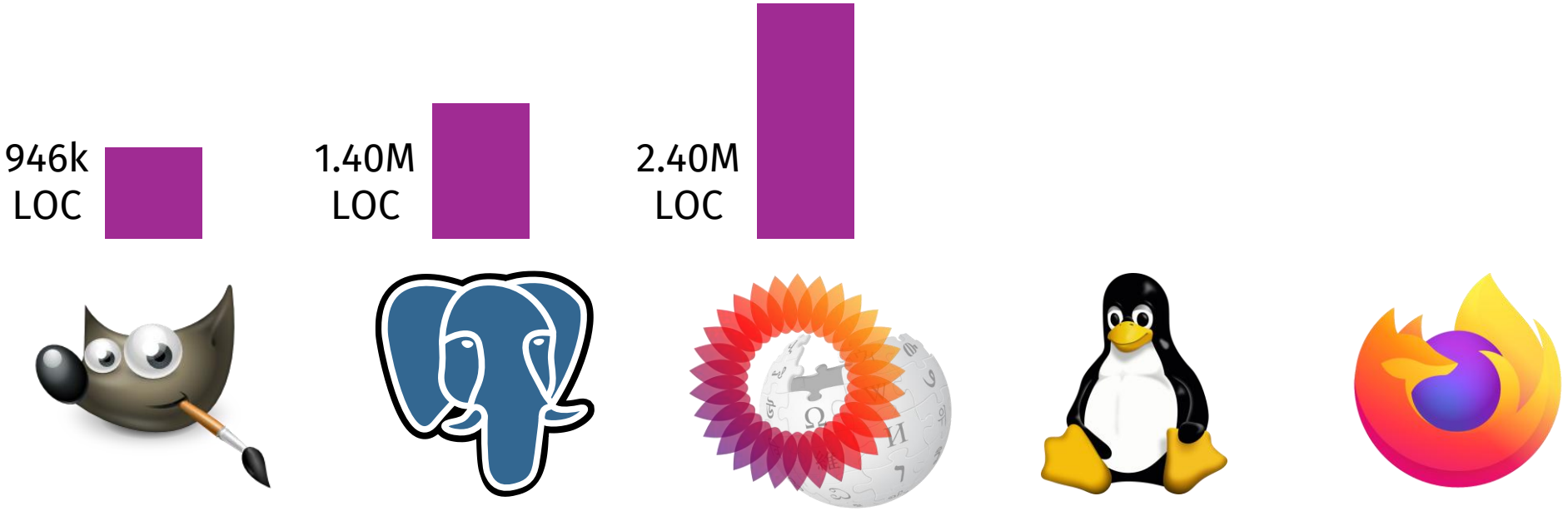
Package and Deploy



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

See also: https://informationisbeautiful.net/visualizations/million-lines-of-code/
Data from January 2026 with https://cloc.info/

The size of modern software



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

See also: https://informationisbeautiful.net/visualizations/million-lines-of-code/
Data from January 2026 with https://cloc.info/

The size of modern software

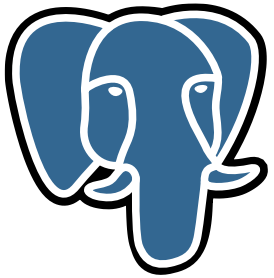
(or why is software buggy)

946k LOC



GIMP

1.40M LOC



PostgreSQL

2.40M LOC



MediaWiki

31.5M LOC



Linux

44.0M LOC



Firefox



LOTR
~460k words

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

They're not even the largest software!
It's a wonder large software work at all.

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Software Engineering



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Definition #1

Software Engineering is the practice of designing, developing, testing, and maintaining software

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0



“As long as there were no machines, programming was no problem at all; when we had a few weak computers, programming became a mild problem; and now we have gigantic computers, programming has become an equally gigantic problem.”

Edsger W. Dijkstra
Communications of the ACM
Volume 15(10), October 1972

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Ariane 5 flight V88



©ESA/CNES/Arianespace



©ESA

Therac-25

tom's HARDWARE

TRENDING RAM Combo Deals Panther Lake

Software

Notorious software bug was killing people 40 years ago — at least three people died after radiation doses that were 100x too strong from the buggy Therac-25 radiation therapy machine

News By Mark Tyson published 22 September 2025



(Image credit: The National Archives, catalog.archives.gov, NAID: 6361754)

Excel autocorrect genetic research

Edition: New Zealand Donate Newsletters Become an author Sign up as a reader Sign

THE CONVERSATION

Academic rigour, journalistic flair

Search analysis, research, academics...

Arts + Culture Environment + Science Business + Economy Health Education Politics + Society

Paste Format

Excel autocorrect errors still plague genetic research, raising concerns over scientific rigour

Published: August 27, 2021 6.12am NZST

Shutterstock

Autocorrection, or predictive text, is a common feature of many modern tech tools, from internet searches to messaging apps and word processors. Autocorrection can be a blessing, but when the algorithm makes mistakes it can change the message in dramatic and sometimes hilarious ways.

Authors

Mark Ziemann

Lecturer in Biotechnology, Bioinformatics, Deakin

Mandhri Abeysooriya

Deakin University

Knight Capital incident

REUTERS EDITION: U.S. SIGN IN | REGISTER Search Reuters

HOME BUSINESS MARKETS WORLD POLITICS TECH OPINION BREAKINGVIEWS MONEY LIFE PICTURES VIDEO

Markets | Wed Aug 1, 2012 6:31pm EDT

Error by Knight Capital rips through stock market

NEW YORK | BY CAROLINE VALETKEVITCH AND CHUCK MIKOLAJCZAK

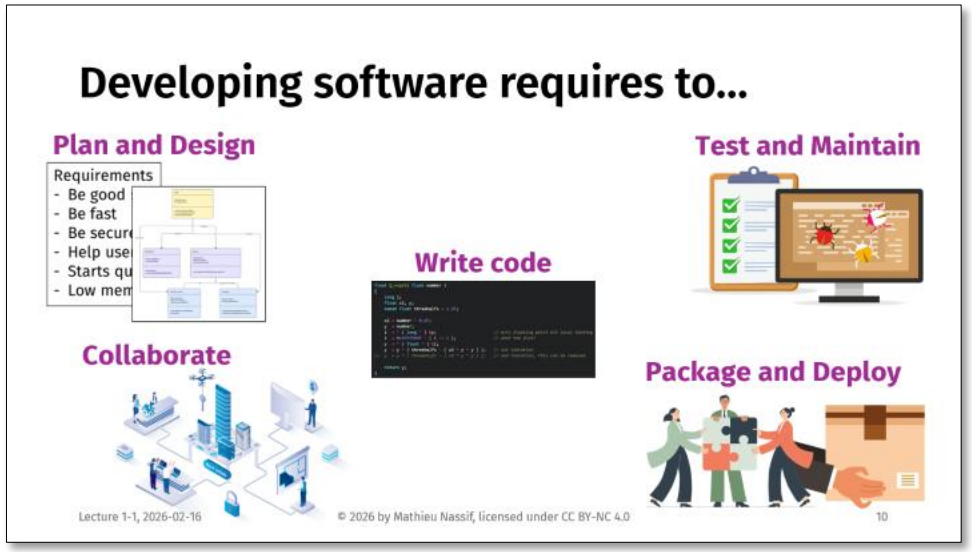
A technology breakdown at a major trading firm roiled the prices of 140 stocks listed on the New York Stock Exchange on Wednesday, undermining fragile invest stability of U.S. stock markets.

The problems at Knight Capital Group Inc (KCG.N), one of the larges sells stocks to provide liquidity to the markets, emerged at the beginn

TRENDING ON REUTERS

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Designing, developing, testing, and maintaining software





Avoid software project failure due to:

- Unmanaged risks and complexity
- Unrealistic, unarticulated goals
- Inaccurate resource estimates
- Poor management or communication
- Sloppy development practices
- Reliance on immature technologies
- Commercial pressures
- Stakeholder politics
- …

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Definition #2

Software Engineering is the “multi-person development of multi-version programs”

(David Parnas, 2011)

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Ariane 5 flight V88



©ESA/CNES/Arianespace

Software from Ariane 4,
adapted to Ariane 5
with new hardware specs,
created to an overflow error

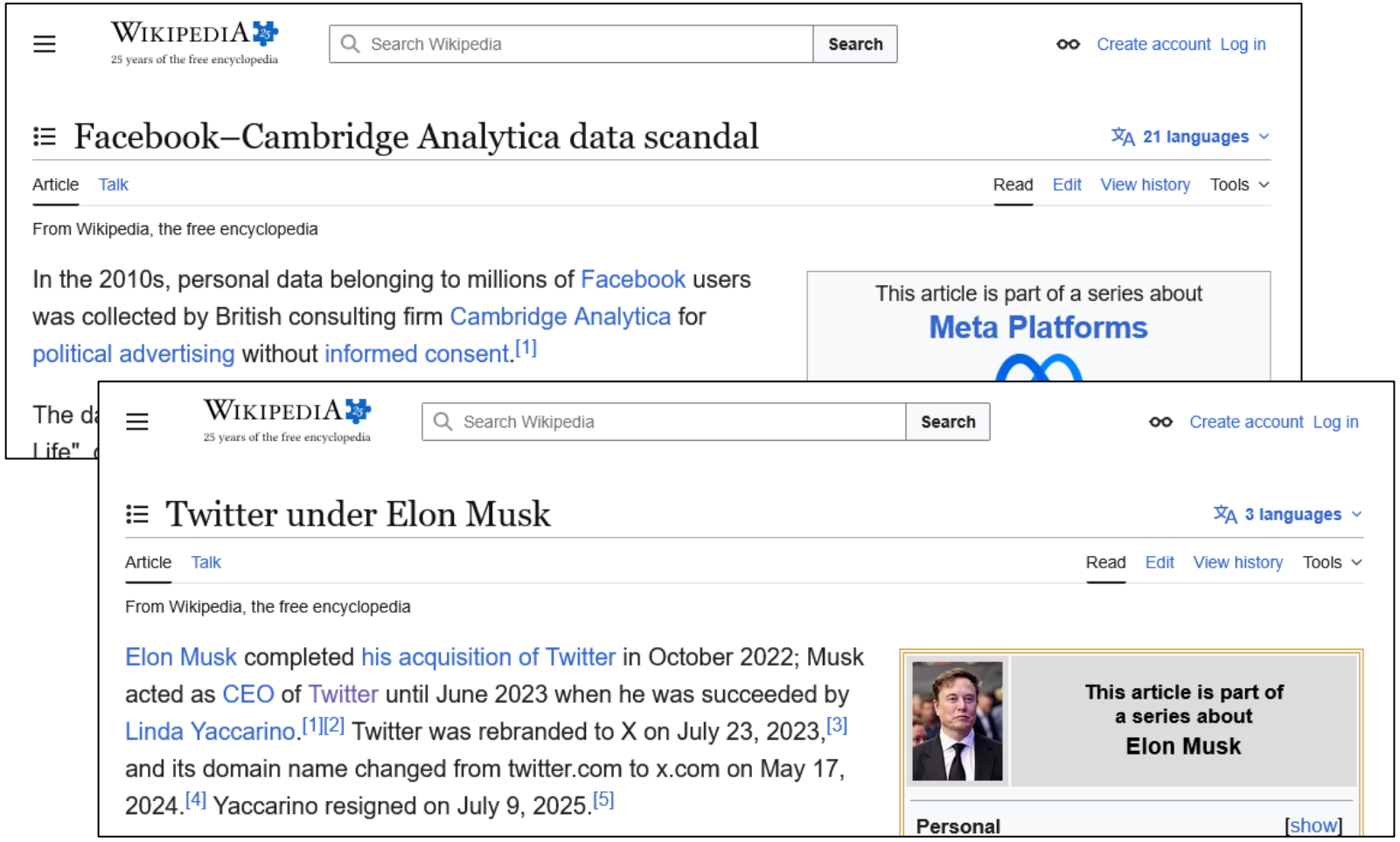
© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Definition #3

Software Engineering is the practice of making software timely, ethical, reliable, reusable, maintainable, evolvable

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Software engineering ethics



Assess and limit harm

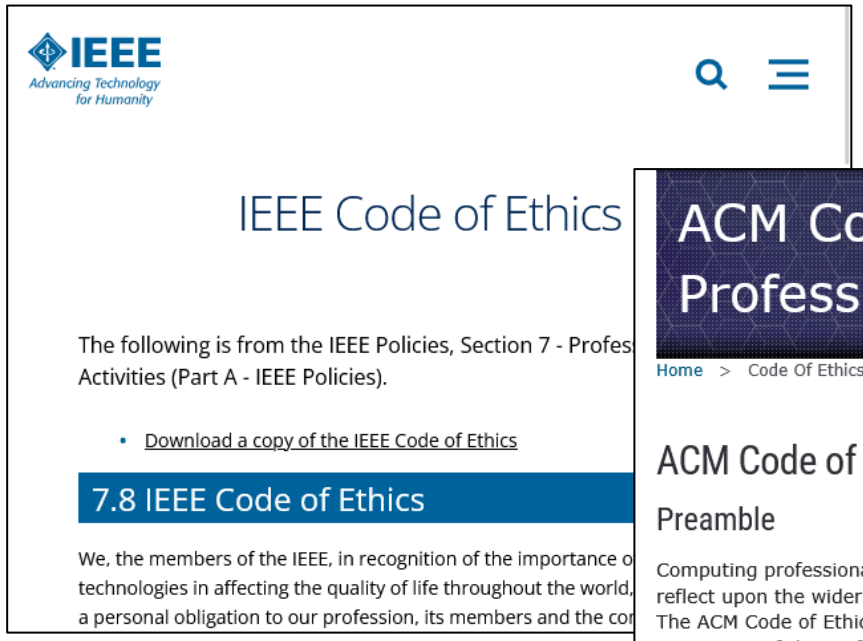
- Privacy and consent
- Safety-critical systems
- Crisis response (e.g., COVID apps)
- Bias in AI-assisted decisions
- Vulnerable users
- Misinformation, manipulation
- …

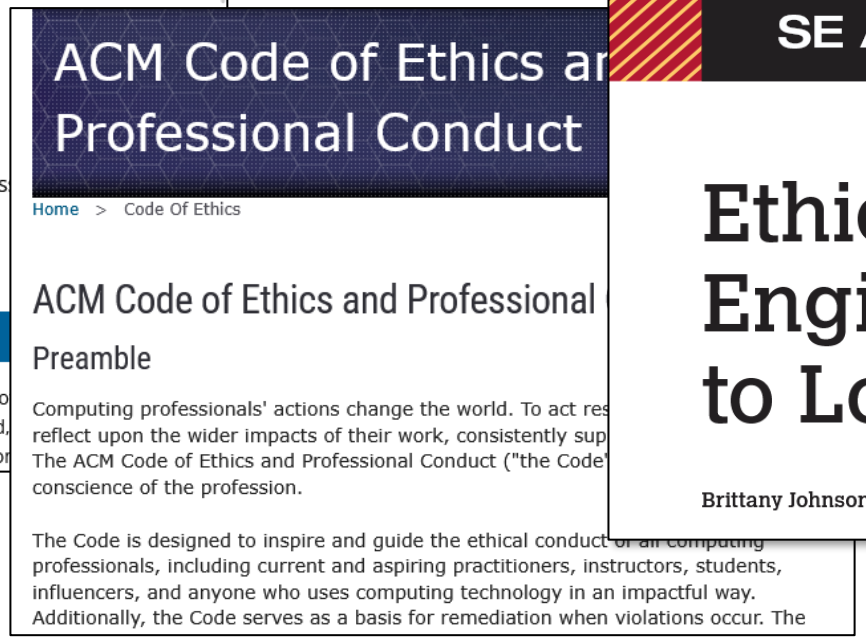
© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Codes of Ethics

https://www.ieee.org/about/corporate/governance/p7-8

Brittany Johnson and Tim Menzies. "Ethics: Why Software Engineers Can't Afford to Look Away". In *IEEE Software*, 41(1), 2024.







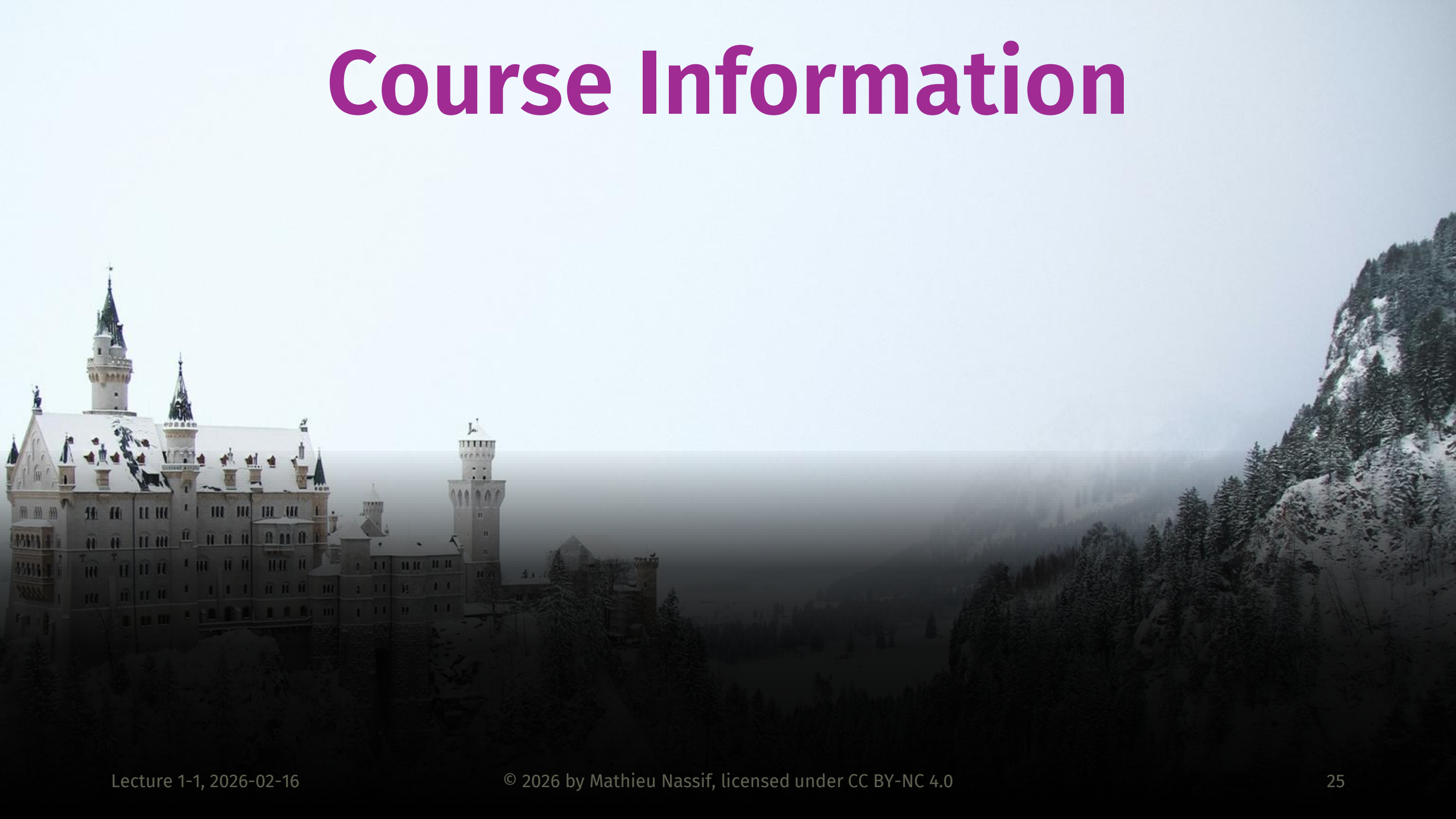
https://www.acm.org/code-of-ethics

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Software Engineering covers all aspects of software production

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Course Information



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

What you will learn

Overview of software engineering

Object-oriented development (with Java)

Analysis and design of software (with UML)

Project management

Engineering tools and techniques

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Your friendly teaching team

Miguel Morales

Course coordinator
JE 210, miguel.morales@canterbury.ac.nz

Mathieu Nassif

Lecturer
JE 310, mathieu.nassif@canterbury.ac.nz

Morgan English

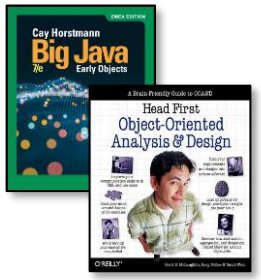
Senior tutor
JE 212, morgan.english@canterbury.ac.nz

Danita Sun, Hannah Grace, Corey Hines, Tannin Heely

Tutors and tech support
tut201@cosc.canterbury.ac.nz

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Course Structure and Assessments



Lectures
- Term 1: Mathieu
- Term 2: Miguel
- Textbooks available from library

Labs and Tutorials
- Self-directed work
- Bonus for completing all

Week 12: Project demos

[20%] Lab Test
- due 20 March
- 90-minute
- computer-based
- closed-book

[30%] Project
- due 22 May
- released before the break

[50%] Final exam

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

How to succeed in this course

- Write code!
- Go beyond “it’s working fine”
- Interact with each other and the teaching staff
  - Don’t hesitate to ask for help or feedback
  - Attend lectures and labs

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

WHO TO SEE WHEN YOU HAVE AN ISSUE IN YOUR COURSE...

LECTURER

- IF YOU CAN'T GET YOUR ASSIGNMENT IN ON TIME OR HAVE AN ASSESSMENT CLASH
- IF YOU ARE FEELING STRESSED ABOUT SOMETHING IN YOUR COURSE
- IF YOU HAVE QUESTIONS ABOUT COURSE CONTENT

CLASS REP*

- IF YOU WANT TO PROVIDE ANONYMOUS FEEDBACK ON AN ASSIGNMENT
- IF YOU WANT TO PROVIDE ANONYMOUS FEEDBACK/ COMMENTS ABOUT YOUR LECTURER

UC STUDENT CARE

- IF YOU NEED PRACTICAL ADVICE REGARDING YOUR WELLBEING AND SUCCESS
- IF YOU ARE FALLING BEHIND IN YOUR STUDIES

UCSA ADVOCACY & WELFARE TEAM

- IF YOU CAN'T FOCUS ON YOUR STUDIES BECAUSE OF FINANCIAL WORRIES
- IF YOU WANT ADVICE OR SUPPORT REGARDING AN ACADEMIC DISPUTE

ACADEMIC SKILLS CENTRE

- IF YOU WANT SOMEONE TO READ OVER YOUR ESSAY
- IF YOU WANT HELP WITH REFERENCING
- IF YOU WOULD LIKE SOME WRITING AND STUDY STRATEGIES

TE AKATOKI

- IF YOU HAVE FEEDBACK OR WANT ADVICE AS A TAUIRA MĀORI
- IF YOU WANT SOCIAL, CULTURAL OR ACADEMIC SUPPORT WITH OTHER TAURIA MĀORI

***NOTE: Class reps are trained to anonymously communicate the feedback they have been given to their lecturers, relevant departments and the UCSA**

FOR MORE OPTIONS YOU CAN VISIT: www.canterbury.ac.nz/support/wellbeing-hub/
www.ucsa.org.nz/student-support

Calling Class Reps!

“students who act as liaisons between the students in a course and the lecturers and Department that runs that course”

Help the student voice be heard and boost your CV

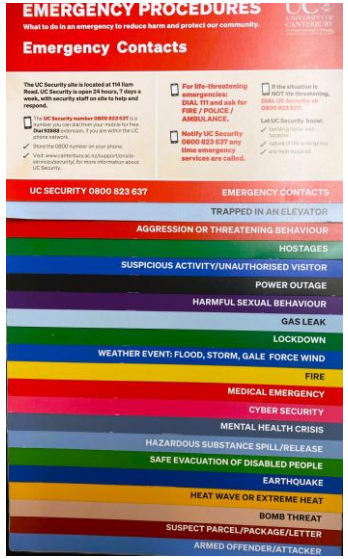
https://ucsa.org.nz/support/classrepresentatives/

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Health and Safety

UC Emergency: 0800 823 637
Dial 111 for police/fire/ambulance

Emergency flip charts

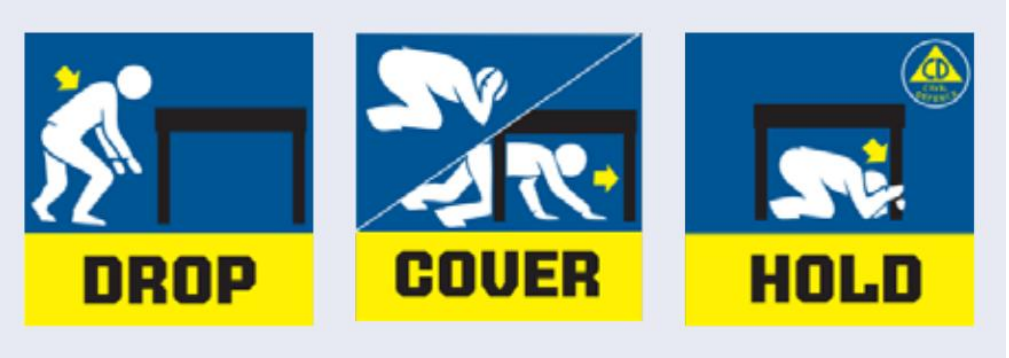


https://www.canterbury.ac.nz/life/support-and-wellbeing/emergency-information/emergency-topics/emergency-procedures

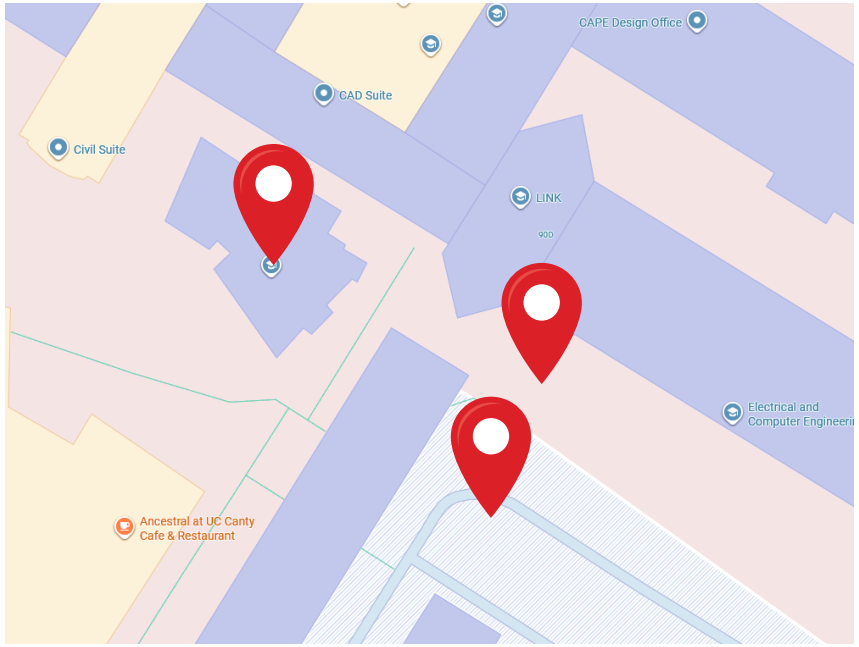
© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Health and Safety

Earthquakes: Drop, Cover, Hold
Once shaking stops, evacuate to Ilam fields or off campus



Fires: Exits and assembly points



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Test Anxiety!

Participate in a study to help a fellow student and develop new skills

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Study on Test Anxiety in CS and SE Education

UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

- **Objectives**
  - **Gain insights** into undergraduate **current experiences with test anxiety in programming tests** and its impact.
  - **Explore the effectiveness of** the proposed **intervention in reducing test anxiety** among CS and SE students.
  - Assess how the intervention influence mental well-being and academic performance.

- **Research team**
  - Hirasha Pooliyadda (PhD student)
  - Valerie Sotardi (School of Curriculum, Pedagogy and Assessment)
  - Miguel Morales

Slides courtesy of Hirasha Pooliyadda

Study timeline

- Three phases, a few surveys, and potentially using the *application*...

Preliminary Survey



Available from 17th February

Experiment Period (Using the application)

Brief Survey I



9th March

Brief Survey II



16th March

Post-experiment Survey



Available from 20th March

Potential benefits

- Gain a better understanding of test anxiety management.
- Learning some important skills, which will be helpful when preparing for the lab test.
- Four (04) students who complete the study will have **the chance to win a $20 voucher** through a raffle.



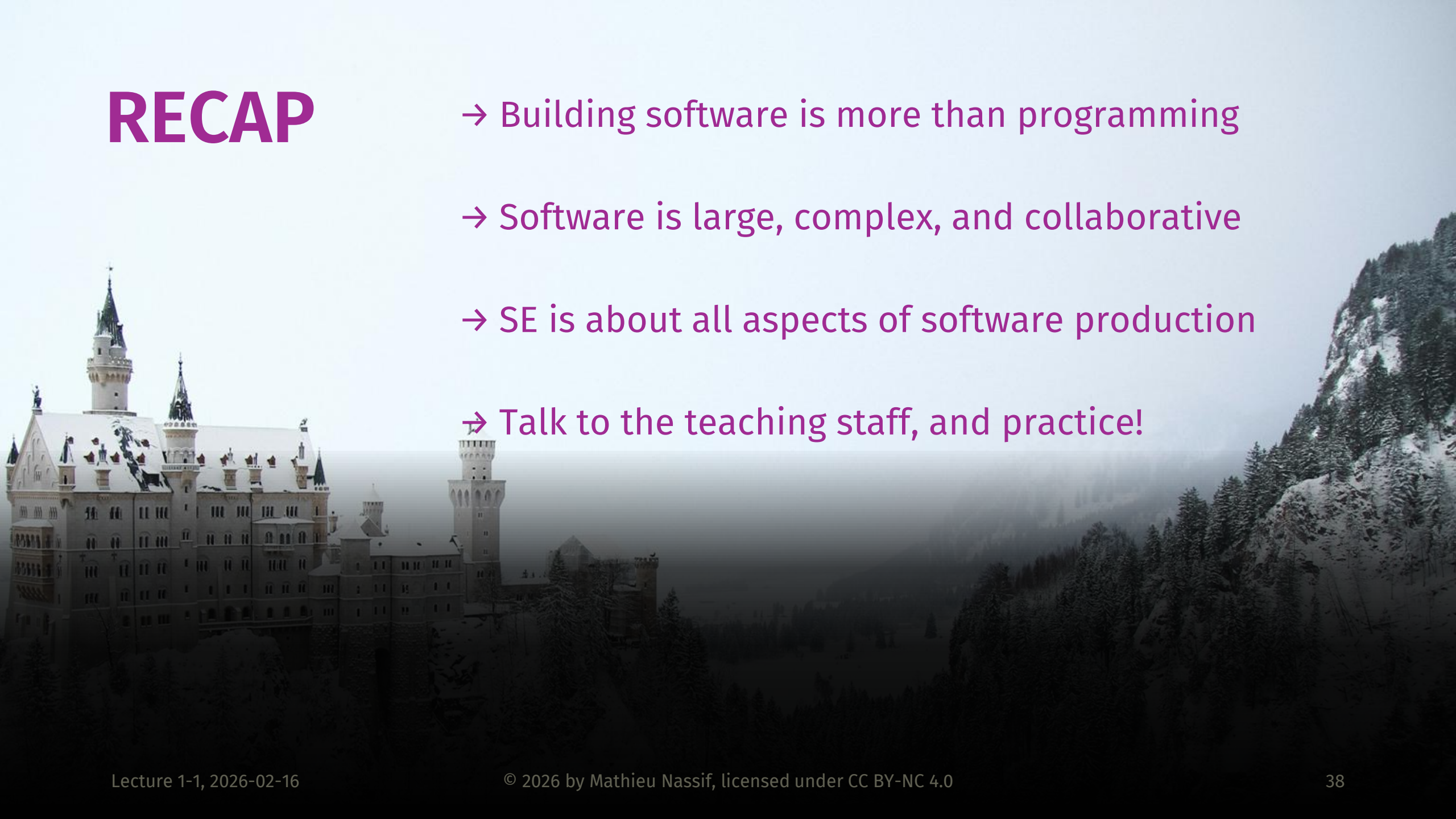
RECAP

→ Building software is more than programming

→ Software is large, complex, and collaborative

→ SE is about all aspects of software production

→ Talk to the teaching staff, and practice!



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0